Analyse de firmware

MCU de contrôle d’un micro-onduleur Deye

Rétro-ingénierie du firmware `MCU1-XXXXWG4-800W-V0319.bin`

Cible	Micro-onduleur Deye G4 800 W (SUN-M G4)
Fichiers analysés	MCU1-XXXXWG4-800W-V0319.bin (71 680 octets, MCU de contrôle) MW4C_14_CSIP_1.14.bin (488 976 octets, module WiFi)
Processeurs	Microchip dsPIC33F (DSC 24 bits) + Andes NDS32 big-endian (module WiFi)
Objet de l’étude	Contrôle de la puissance active : RAM vs EEPROM Fréquence de rafraîchissement des mesures Modbus Surface de commande à distance (Solarman) et sécurité
Outils	Ghidra 12.1.4 (module PIC/dsPIC), scripts d’analyse Java
Date	28/10/2026

Document technique — rétro-ingénierie à des fins d’interopérabilité et de recherche sur matériel possédé par l’utilisateur.

1. Résumé exécutif

Ce rapport documente la rétro-ingénierie du firmware du microcontrôleur de contrôle d'un micro-onduleur Deye (modèle G4 800 W). Contrairement au module de communication WiFi (SoC Andes NDS32 big-endian, SDK « Tiramisu »), ce firmware pilote directement l'étage de puissance et implémente la pile Modbus RTU par laquelle l'onduleur est commandé — et par laquelle il peut être **arrêté à distance** (§ 10). Une troisième partie (§ 11 à 12) explique pourquoi les mesures ne se rafraîchissent que toutes les ~5 minutes.

Principales découvertes :

- Le MCU est un **Microchip dsPIC33F** (24 bits). Le firmware gère le PWM de puissance, les mesures ADC, deux UART et un **EEPROM externe I²C à l'adresse 0x51**. Aucune écriture de la flash interne.
- Toute la persistance passe par cet EEPROM, et chaque écriture est **conditionnée par un *memcmp*** : le firmware ne réécrit une page que si son contenu a changé (limitation naturelle de l'usure).
- La table de dispatch des écritures de registres Modbus a été entièrement cartographiée.
- Le registre **0x28** (puissance active %) écrit une variable vive `0x0A22` qui est **persistée** (restaurée au démarrage depuis l'EEPROM).
- **Découverte clé :** le registre **0x35** écrit la variable `0x0A24`, qui alimente la **même chaîne de référence de puissance** que 0x28 mais **n'est pas persistée en EEPROM** — c'est un levier de modulation de puissance **volatil** (sans usure).
- Le registre **0x37** (0x09F4) touche aussi à la puissance mais comme facteur d'échelle (nominal 10000) régulé par le firmware lui-même : peu fiable comme consigne utilisateur.
- **Rafraîchissement des mesures :** le module WiFi relaie les requêtes Solarman V5 **en direct** (aucun cache). C'est le dsPIC qui ne recalcule ses registres de mesure (puissance, tension, courant, fréquence, température, PV) **que toutes les ~302 s**, et la puissance publiée est une **moyenne glissante sur 10 minutes**. Constantes codées en dur, non modifiables par Modbus.
- **Commande à distance (§ 10) :** le module WiFi relaie sans filtrage les trames reçues du cloud Solarman vers le dsPIC — **lecture et écriture de tout registre**, et même **mise à jour du firmware** de l'onduleur. L'écriture **FC16 du registre 0x2B = 2** provoque un **arrêt immédiat et persistant** de la production (déconnexion + défaut, conservé après coupure secteur). La seule « authentification » est le numéro de série du logger, transmis **en clair** : efficace contre un scan aveugle, nul contre un acteur sur le chemin réseau (MITM).
- **Pas de backdoor caché sans authentification :** le passthrough brut `AT+INVDATA` (sans contrôle de série) existe dans le binaire mais est **du code mort** (branche jamais atteinte) ; aucun serveur TCP local n'est présent dans ce firmware.

2. Contexte et objectif

Un micro-onduleur Deye est commandé via le protocole Solarman (encapsulant du Modbus RTU) à travers son module WiFi. L'objectif de l'étude était de déterminer si la modification de la puissance active de sortie passe obligatoirement par une écriture en EEPROM (donc avec usure à long terme), ou s'il existe un registre / un chemin permettant de changer cette puissance en RAM uniquement — utile pour une modulation fréquente (suivi de consommation, pilotage zero-export) sans dégrader l'EEPROM.

3. Identification du firmware

L'architecture a été identifiée par la structure du binaire, sans se fier au nom de fichier :

- **Octet « phantom » à 0** tous les 4 octets (97,5 % des positions) : signature des instructions 24 bits de la famille PIC24 / dsPIC (chaque mot d'instruction occupe 4 octets dont un octet nul).
- **Table des vecteurs** en début d'image : vecteur de reset à PA0, puis table d'interruptions de `GOTO` (opcode 0x04) dès PA0x04. Handler par défaut en PA0x44D2 ; vecteurs spécifiques (timers / ADC / PWM) en PA0x3CEE, 0x3D10, 0x406E.
- **Séquences de déverrouillage OSCCON** (0x46/0x57 et 0x78/0x9A) et registre PLLFBD en 0x0746 : confirment la variante **dsPIC33F** (et non 33E/33C).
- Taille utile : ~52 Ko de code (17 920 mots d'instruction) — cohérent avec une flash de 64 Ko.

En-tête et vecteurs (extraits) :

```
reset @PA0 : GOTO<br/>IVT @PA0x04.. : GOTO 0x0044D2 (handler par defaut, x60)<br/>vec timers/ADC/PW
M : GOTO 0x003CEE / 0x003D10 / 0x00406E<br/>Langage Ghidra : dsPIC33F:LE:24:default
```

4. Cartographie mémoire et périphériques

Comptage des accès aux registres de périphériques (SFR) dans le code désassemblé :

Périphérique	Plage SFR	Accès	Rôle
PWM	`0x0400-0x04FF`	`399`	Pilotage de l'étage de puissance (commutation)
ADC1	`0x0300-0x033F`	`30`	Mesures tension / courant (PV, réseau)
I²C1	`0x0200-0x020E`	`32`	EEPROM externe 24Cxx @ esclave 0x51 (persistance)
UART1 / UART2	`0x0220-0x023F`	`19 / 12`	Lien Modbus (vers module WiFi) + debug
Ports E/S	`0x02C0-0x02FF`	`99`	GPIO (relais, signaux)
SPI	`0x0240-0x026F`	`0`	Non utilisé
Flash interne (NVM)	`0x0760-0x0766`	`0`	**Jamais utilisée** : pas d'auto-programmation flash

Conclusion : la flash interne du dsPIC n'est jamais reprogrammée (registres NVMCON/NVMKEY jamais référencés). **Toute donnée non volatile est stockée dans l'EEPROM I²C 0x51**, le même composant que sur les autres onduleurs Deye analysés.

5. Pile de communication Modbus RTU

Le MCU traite directement les trames Modbus RTU reçues sur l'UART :

- Routine **CRC16** (polynôme 0xA001) localisée en PA0x57D2.
- Constructeur de trame de réponse en PA0x5802 : tampon en RAM à `0x0B02`, embarque le **Device Type 0x201**, puis ajoute le CRC.
- Tampon de trame reçue en RAM à `0x0DD8` ; le dispatcher valide le CRC, puis traite les codes fonction (lecture 0x03, écriture 0x06 / 0x10).

6. Mécanisme de persistance EEPROM (change-gated)

Point central du rapport. Les écritures EEPROM ne sont **jamais** faites directement par le chemin temps réel de traitement Modbus : elles sont exécutées de manière découplée, et chacune des 7 sites d'écriture identifiés suit exactement le même schéma :

```
rcall 0x81A2   ; LECTURE de la page de 32 octets depuis l'EEPROM (esclave 0xA2 = 0x51)
mov.w #0x20,W2 ; 32 octets
rcall 0x2AC4   ; memcmp(bloc_courant, bloc_EEPROM, 32)
cp0.w W0
bra z, suite   ; si IDENTIQUE -> on saute l'ecriture
...
rcall 0x8112   ; ECRITURE page EEPROM (routine 0x10224) uniquement si different
```

La routine d'écriture (PA0x8112) écrit des **pages de 32 octets** depuis un pointeur RAM vers l'EEPROM (pages 0x1A à 0x1D observées). La configuration est relue de l'EEPROM au démarrage.

Conséquence pratique :

- Réécrire la **même** valeur -> **aucune écriture EEPROM** (0 usure).
- Écrire une **nouvelle** valeur -> **une seule** écriture de page de 32 octets.
- Une modulation haute fréquence à valeurs toujours différentes resterait usante à long terme (endurance typique d'une 24Cxx : ~1 million de cycles par page).

7. Table de dispatch des écritures de registres

Le dispatcher d'écriture utilise une table de saut calculé (`bra W4` en PA0x6040), indexée par `(numéro_de_registre - 0x15)`, couvrant les registres 0x15 à 0xBB (167 entrées). Deux classes de handlers se distinguent :

Classe A — réglages persistés (écriture directe en image RAM 0x1200-0x129x, mirroirée EEPROM)

Registre	RAM	Registre	RAM
0x1B	0x1244	0x1C	0x1248
0x1D	0x124E	0x1E	0x1252
0x1F	0x1272	0x20	0x124A
0x22	0x122A	0x23	0x122E
0x24	0x1234	0x25	0x122C
0x26	0x1200	0x29	0x1236
0x2C	0x1230	0x2D	0x1232
0x2E	0x1224	0x2F	0x1226
0x31	0x1228	0x96-0xB2	0x1274-0x129x

Classe B — variables vives (zone 0x09xx / 0x0Axx, non directement dans l'image de réglages)

Registre (déc)	RAM	Nature
`0x28 (40)`	`0x0A22`	**Puissance active %** (0-100), lue par la boucle de contrôle, avec rampe. Persistée.
`0x35 (53)`	`0x0A24`	**Puissance active (centi-%)**, alimente la même référence 0x0A26. **Volatile.**
`0x37 (55)`	`0x09F4`	Facteur d'échelle (nominal 10000) régulé par le firmware. Volatile mais instable.
`0x2B/0x30/0x33`	`0x092A/0x0922/0x0924`	Paramètres vifs divers.
`0x34/0x36/0x39`	`0x0A0E/0x0980/0x0800`	Paramètres vifs divers.
`0x32 / 0x38`	`0x09D8 / 0x09EA`	Paramètres vifs divers.

Les registres 0x3A à 0x95 (et 0x2A, 0xAA, 0xB3-0xBB) pointent vers le handler par défaut (0xC5C4) : ce sont des entrées réservées / sans effet.

8. Analyse de la commande de puissance active

8.1 — Registre 0x28 (variable 0x0A22) : la consigne persistée

Le code de contrôle traite `0x0A22` comme un **pourcentage 0-100** de puissance active :

```
0x7EE0 : lit 0x0A22 ; (0x0A22 + offset) compare a 0x64 (=100) ; 100 - W0  -> clamp a 100%
rampe  : dec 0x0A22 ; cmp 0x77 (=119) ; sinon 0x0A22 = 0x64 (=100)   -> montee/descente bornee
0x7AC8 : 0x0A22 -> 0x0A26 ; puis 0x0A26 x accumulateur (0x1054..0x105A) = reference de puissance
```

La variable est restaurée depuis l'EEPROM au démarrage (écriture en PA0x48B2), ce qui explique que la valeur réglée « survive » à l'extinction nocturne de l'onduleur. C'est le registre de modulation officiel, avec usure EEPROM limitée par le *memcmp*.

8.2 — Registre 0x35 (variable 0x0A24) : le levier VOLATIL — découverte clé

> **0x0A24 alimente la MÊME chaîne de référence de puissance que 0x0A22** (destination commune `0x0A26`), mais en **centi-%** (résolution plus fine), et **n'est PAS restaurée depuis l'EEPROM au démarrage** : c'est une **variable vive (RAM) — modulation de puissance sans usure EEPROM.**

```
0x7AAC : W0 = 0x0A24 ; W0 / 0x64 (=100)  -> 0x0A26   (0x0A24 est donc en centi-pourcent)
0x7AC8 : chemin alternatif ; 0x0A22 -> 0x0A26        (selection a l'execution en 0x7AA8)
```

Nuance à valider sur le terrain : un sélecteur (branchement conditionnel en PA0x3D54 / instruction `bra c`) choisit à l'exécution entre le chemin 0x0A22 et le chemin 0x0A24 selon un état interne. L'effet de 0x35 n'est donc pris en compte que lorsque ce sélecteur pointe sur 0x0A24 — d'où l'importance du test empirique ci-après.

8.3 — Registre 0x37 (variable 0x09F4) : facteur d'échelle, pas une consigne propre

- Plage centrée sur **10000** (0x2710), bornée à 10000 (et non 0-100) : il s'agit d'un **facteur d'échelle / référence per-unit** (10000 = 100,00 %), pas d'un plafond en %.
- Utilisé comme multiplicateur dans plusieurs calculs (x mesure, /75, /78).
- **Réécrit par le firmware lui-même** (recalculé, remis à 10000 hors borne) : une valeur écrite par Modbus peut être écrasée en quelques cycles. Cohérent avec le label « factory only » de la documentation Deye.
- Non restauré depuis l'EEPROM : volatile, mais peu exploitable comme consigne stable.

9. Module WiFi MW4C : relais Solarman V5 sans cache

Correction d'identification : le module WiFi n'est pas un Marvell 88MW300 (ARM). Les mots de code `46 xx xx xx / 58 xx xx xx` sont des paires **sethi/ori Andes NDS32** big-endian ; 459 références vers des chaînes tombent juste si le payload (offset fichier 0x100) est chargé en **0x30008000**. SoC de type iComm SV6266 (SDK « Tiramisu », mots de passe `icomm`…). Projet Ghidra : `C:\Tools\nds32-mw4c` (NDS32:BE:32, 1537 fonctions décompilées).

Traitement d'une requête V5 locale (parseur `FUN_ram_3005282c`)

```
1. cherche 0xA5, verifie longueur, octet de fin 0x15, checksum et numero de serie du logger
2. rx[11] = type de trame : 2 = Modbus transparent, 3 = commande AT, 1 = synchro heure
3. type 2 : copie la trame Modbus (payload + 15)
   FUN_ram_3004cfe4 : attend que l'UART soit libre (<= 199 x 10 ms), vide le tampon RX
   FUN_ram_3002d348(1, trame, len, 1000) : envoi sur UART1 vers le dsPIC
   FUN_ram_3004d340 : attend la reponse (99 x 10 ms ~ 1 s)
   FUN_ram_3004ed60 : renvoie la reponse encapsulee en V5 (ou code d'erreur 5 si timeout)
```

Le module n'a **ni cache ni limite de fréquence** sur ce chemin : chaque requête d'un ESP32 atteint le dsPIC. Les timers internes du module (30 s, 60 s, 300 s, 12 h) ne servent qu'au cloud Solarman (`Deye_Time60S_Read_Inverter_Data`, upload, synchro horaire). Seul effet possible : une attente jusqu'à ~2 s si le module interroge lui-même le dsPIC au même moment.

10. Surface de commande à distance et sécurité (passerelle Solarman)

Le parseur V5 qui relaie les lectures (§ 9) relaie aussi les **écritures** et les commandes de **mise à jour**. Le cloud Solarman — ou tout acteur placé sur le chemin réseau — peut donc agir directement sur l'onduleur. Cette section décrit ce qui est réellement atteignable et avec quelle authentification. Analyse menée sur le firmware du module WiFi (NDS32, projet `C:\Tools\nds32-mw4c`).

10.1 — Un seul socket actif, authentifié par le numéro de série

Le module n'expose qu'**une seule liaison de données active** : le **socket B**, client TCP du cloud m2m Solarman (callback `socketb_recv_callback` = `FUN_ram_30048f90`, flag de connexion `gp+0xAC5B`, routine `YZ_TCPB_ShutDown`, portail `AP_SOLAR_PORTAL_M2M`). Toute trame, lecture *comme* écriture, passe par le même parseur `FUN_ram_3005282c`, qui vérifie le **numéro de série du logger** :

```
FUN_ram_3004d0ac(rx+serial)  vs  gp+0xB2A4   ;  si different -> trame rejetee
```

Ce n'est donc **pas** un accès anonyme : il faut connaître le numéro de série. Mais ce numéro est **imprimé sur l'étiquette**, connu du cloud, et surtout transmis **en clair dans chaque trame TCP** (aucun TLS). Il est donc récupérable par simple écoute, et un attaquant en **homme du milieu** peut forger des trames valides. L'« authentification » bloque un scan Internet aveugle, pas un acteur sur le chemin réseau.

10.2 — Le passthrough brut `AT+INVDATA` est du code mort

Le parseur comporte une seconde branche (socket A, tampon `gp+0xC9AC`) qui appelle `FUN_ram_3004f644` — un passthrough brut `AT+INVDATA` recopiant des octets vers l'UART de l'onduleur **sans aucun contrôle de série**. Vérification exhaustive du binaire : son flag de garde `gp+0xAC58` n'est **jamais** mis à une valeur non nulle (uniquement testé à 0 puis remis à 0), `gp+0xAC64` (longueur) jamais écrit, et un seul callback de réception recopie des données (vers le socket B). **Cette branche est donc inatteignable à l'exécution** : vestige de la base générique Hi-Flying (qui supporte SOCKA + SOCKB), dont le build Deye ne câble que SOCKB.

De plus, **aucun serveur TCP local** n'est présent (pas de `listen`, pas de port 8899). L'accès LAN direct à l'onduleur par passerelle transparente — présent sur d'anciens firmwares de logger — n'existe pas ici ; l'accès local se limite au portail HTTP en mode point d'accès et aux commandes AT sur l'UART.

10.3 — Ce que la passerelle permet réellement

Trame (rx[11])	Effet sur l'onduleur
type 2 / 3	Modbus transparent : **lecture ET écriture de tout registre**, recopié tel quel vers le dsPIC, aucun filtrage de registre.
type 1	Mise à jour firmware de l'onduleur (`Asd_Send_UpgradeMCU`, `LPB100_Inverter_UPDATA`, contrôle CRC32) : **reflash du MCU de puissance**.

Arrêt immédiat et persistant : registre 0x2B

Le registre Modbus **0x2B** correspond à la variable dsPIC `0x092A`, un **code d'état marche/arrêt** chargé de l'EEPROM au démarrage (borné à 1–3). Sa valeur pilote directement la production :

```
main PA 0x50E4 : si 0x092A appartient a {2,3} -> bit 'produire' (0x09DA b0) force a 0  (pas de coup
lage)
           PA 0x5?? : chemin deconnexion/defaut  rcall 0x848C(1/2/3) + code defaut 4 empile
boot          : 0x092A relu de l'EEPROM, borne 1-3 ;  valeur 1 = marche normale
mapping confirme : image reponse FC03  0x092A -> 0x0B62 ;  (0x0B62-0x0B0C)/2 = 43 = reg 0x2B
```

Comme `0x092A` est **persisté en EEPROM**, une écriture **FC16 reg 0x2B = 2 (ou 3)** coupe la production **immédiatement** et l'onduleur **reste à l'arrêt après une coupure secteur**, jusqu'à réécriture de la valeur 1. C'est le « kill à distance » le plus net — plus radical que le bridage en % via 0x28 / 0x35, qui ne fait que moduler la puissance. Réserve : les libellés exacts 1/2/3 sont déduits du câblage logique (pas d'une chaîne de texte) et méritent une validation sur l'appareil.

10.4 — Réduction du risque

- Bloquer en sortie les hôtes cloud Solarman (`solarmandata.com`, `data2.solarmanpv.com`, IP `115.29.186.234` / `176.58.117.69`) et isoler le logger dans un VLAN sans accès sortant libre.
- Ne pas considérer le numéro de série comme un secret : tout circulant en clair, un MITM sur le chemin réseau peut forger des commandes valides (écriture de registre, reflash).
- Bilan : **aucun backdoor caché sans authentification** n'a été trouvé ; le risque vient de la **faiblesse de l'authentification** (série en clair) combinée à la **puissance de la passerelle** (écriture de tout registre + reflash du MCU).

11. Pourquoi les mesures ne bougent que toutes les ~5 minutes

À chaque lecture FC03, le dsPIC (`FUN_rom_005d8e`) efface puis reconstruit l'image des registres en RAM 0x0B0C (registre n = 0x0B0C + 2n) via `FUN_rom_0058d6`. Cette image est remplie à partir de variables « d'affichage » qui, elles, ne sont mises à jour que périodiquement.

11.1 — Base de temps : Timer4 à ~100 ms

```
Horloge : FRC 7,37 MHz + PLL (PLLFBD = 0x29 -> M = 43, PLLPRE/PLLPOST -> N1 = N2 = 2)
          Fosc = 7,37 x 43 / 4 = 79,2 MHz  ->  Fcy = 39,6 MHz
Timer4  : T4CON |= 0x30 (prescaler 1:256), PR4 = 0x3CB9 (15545)   [PA 0x7D94]
          periode = 15546 x 256 / 39,6 MHz = 100,5 ms
ISR T4  : PA 0x1F52 (efface T4IF = IFS1<11>)
```

11.2 — Slot d'une minute et fenêtre glissante de 10 minutes

```
ISR T4 : inc 0x08EC ; si > 600 (mov #0x258 @PA 0x1F90)  -> toutes les 601 ticks = 60,4 s
         0x08DC <- accu energie voie 1 (0x08D4), 0x08E0 <- accu voie 2 (0x08D8), raz accus
         flag 0x08EE = 1
Accus  : PA 0x096A : a chaque cycle reseau, 0x08D4 += 2 x P_inst voie 1 (0x0F7C), idem voie 2 (0x0F
80)
Main   : PA 0x50E4 : si flag -> decale 2 tableaux de 10 slots (0x0AAE.., 0x0AD6..)
         somme = somme - plus ancien + nouveau
         P_voie (0x0A8E / 0x0A92) = somme / 2 / 30000 / 75   (50 Hz)  [#0x7530 @PA 0x5724]
                                  = somme / 2 / 36000 / 75   (60 Hz)  [#0x8CA0 @PA 0x5756]
```

30000 = 600 s × 50 cycles/s : la puissance par voie est donc la **moyenne des 10 dernières minutes**, recalculée chaque minute.

11.3 — Publication seulement un slot sur cinq

```
PA 0x50E8 : inc 0x08EF ; PA 0x5116 : sub.b W0,#5 ; bra nz -> chemin court (fenetre seule)
si 0x08EF == 5 (toutes les ~302 s) : raz 0x08EF et recalcul de toutes les valeurs publiees
```

Registre	Variable RAM	Contenu	Rafraîchi
`0x56 / 0x57`	`0x094E / 0x0950`	Puissance AC (moyenne 10 min)	~302 s
`0x4C`	`0x0F2A`	Courant AC (= P x 10 / V)	~302 s
`0x49`	`0x1064`	Tension AC	~302 s
`0x4F`	`0x0F2C`	Fréquence	~302 s
`0x5A`	`0x0952`	Température radiateur	~302 s
`0x6D-0x74`	`0x0966.. / 0x096E..`	Tensions / courants PV	~302 s
`0x3C, 0x3F/40`	énergie (EEPROM page 0x1C)	Production jour / totale	~302 s

Ni la puissance instantanée (0x0F7C/0x0F80) ni la puissance par voie à la minute (0x0A8E) ne sont recopiées dans l'image Modbus. Les compteurs 0x08EC et 0x08EF ne sont touchés que par l'ISR et la boucle principale : **aucun registre Modbus ne permet de changer cette cadence**.

La période de ~302 s découle directement des constantes du firmware (slot de 600 ticks à ~100 ms, publication 1 slot sur 5), gate de publication confirmé par désassemblage (`sub.b W0,#5` @PA 0x5116). Une estimation initiale plus courte (~90–120 s) n'était pas fiable et ne remet pas en cause cette valeur. Calcul d'horloge fondé sur le FRC nominal (7,37 MHz). Test de contrôle : lire 0x49/0x56/0x6D toutes les 5 s et horodater les changements de valeur.

12. Pistes de contournement

12.1 — Mesure externe (recommandée)

Pince ou compteur sur la sortie AC du micro-onduleur (JSY-MK-194/333, Shelly EM, PZEM…) : puissance réelle à la seconde, sans toucher à l'onduleur. Pour une régulation (zéro injection, routeur), c'est la seule source fiable : même patché, le dsPIC ne fournirait que des moyennes.

12.2 — Patch du firmware dsPIC (faisable, non trivial)

Rôle	Instruction (PA)	Offset fichier	Mot actuel
Durée d'un slot (600 ticks)	`mov #0x258,W1 (0x1F90)`	`0x3F20`	`0x202581`
Publication tous les 5 slots	`sub.b W0,#5 (0x5116)`	`0xA22C`	`0x504FE5`
Diviseur fenêtre 50 Hz	`mov #0x7530 (0x5724)`	`0xAE48`	`0x275303`
Diviseur fenêtre 60 Hz	`mov #0x8CA0 (0x5756)`	`0xAEAC`	`0x28CA03`

Changer une seule constante ne suffit pas, à cause de trois effets de bord :

- **Énergie calée sur 5 min :** 0x094E est relu en PA 0x548C / 0x5526, dans le bloc « 1 slot sur 5 », pour incrémenter la production du jour et totale. Passer le test à #1 compterait l'énergie 5 fois.
- **Usure EEPROM :** le même bloc réécrit la page énergie (0x1C), dont le contenu change à chaque passage. Toutes les 6 s, une 24Cxx (~1 M cycles) serait usée en ~2 mois. Il faut séparer recalcul de l'affichage (rapide) et persistance (lente).
- **Fenêtre :** pour la puissance de la dernière minute, utiliser directement le dernier slot (0x08DC/0x08E0) avec un diviseur /10 au lieu de la somme de 10 slots.

Patch propre envisagé : slot réduit (par ex. 6 s, #0x3C), affichage recalculé à chaque slot, énergie et EEPROM conservées à ~5 min, diviseurs ajustés : quelques dizaines d'instructions, après analyse fine du bloc énergie (PA 0x52AE-0x5616).

Flashage : soit par ICSP (PICkit sur la carte ; image complète et bits de protection à vérifier), soit par le module WiFi, qui sait mettre à jour l'onduleur (`AT+INVUPURL`, `yz_inverter_updata`, contrôle CRC32) — protocole et bootloader dsPIC non encore analysés. **Risques :** carte inutilisable si le flash échoue, fin de garantie, perte de la conformité à la certification réseau (même si le patch ne touche que le reporting).

13. Recommandations

Protocole de validation sur l'appareil (écriture en FC16, FC06 étant rejeté) :

Registre	Hypothèse	Test
`0x35 (53)`	% puissance **volatil**, fine (centi-%)	Écrire 5000 -> la puissance bouge ? -> couper/rallumer -> si retour au défaut = levier RAM confirmé.
`0x28 (40)`	% puissance persisté	Écrire 77 -> vérifier qu'il survit au redémarrage (référence).
`0x37 (55)`	facteur x10000 auto-régulé	Écrire une valeur -> vérifier si elle tient >10 s ou si le firmware l'écrase.

Pour une intégration ESPhome (composant `deyemi`), le registre **0x35** est le meilleur candidat pour un pilotage de puissance à haute fréquence sans usure EEPROM, sous réserve de la validation du sélecteur (§ 8.2). Conserver 0x28 pour une consigne persistante « par défaut », et utiliser 0x35 pour la modulation dynamique.

Pour suivre la production en temps réel, ne pas compter sur les registres de mesure du G4 (5 min, lissés 10 min) : ajouter une mesure AC externe (§ 11.1).

14. Annexe — adresses et méthodologie

Élément	Adresse / valeur
Base image / reset	`PA 0x0 (offset fichier = PA x 2)`
Handler IVT par défaut	`PA 0x44D2`
CRC16 Modbus (0xA001)	`PA 0x57D2`
Constructeur réponse Modbus	`PA 0x5802 (tampon RAM 0x0B02)`
Tampon trame reçue	`RAM 0x0DD8`
Table de saut d'écriture	`PA 0x6040 (bra W4), base registre 0x15`
Routine écriture EEPROM	`PA 0x8112 (pages de 32 o)`
EEPROM I²C	`esclave 0x51 (octet de contrôle 0xA2)`
Puissance active (0x28)	`RAM 0x0A22 (%) -> ref. 0x0A26`
Puissance active (0x35)	`RAM 0x0A24 (centi-%) -> ref. 0x0A26`
Référence puissance effective	`RAM 0x0A26`
Registre 0x37	`RAM 0x09F4 (facteur, nominal 10000)`
Dispatcher Modbus / lecture FC03	`PA 0x5D8E ; image registres RAM 0x0B0C (PA 0x58D6)`
ISR Timer4 (tick ~100 ms)	`PA 0x1F52 ; init PA 0x7D94 (PR4 0x3CB9, 1:256)`
Slot 60 s / flag	`compteur 0x08EC (> 600), flag 0x08EE`
Publication 1 slot sur 5	`compteur 0x08EF, test PA 0x5116`
Fenêtre glissante 10 slots	`0x0AAE.. / 0x0AD6.. ; sommes 0x08E4 / 0x08E8`
MW4C : parseur V5	`FUN_ram_3005282c (base 0x30008000, NDS32 BE)`

Méthodologie

- Désassemblage avec Ghidra 12.1.4, langage `dsPIC33F:LE:24:default` ; projet conservé dans `C:\Tools\dspic-mw4c`.
- Import binaire brut à la base 0 ; désassemblage forcé sur tout le bloc code, puis scripts d'analyse Java (table des vecteurs, histogramme SFR, suivi des références, cartographie de la table de dispatch, comparaison des chemins de données).
- Les numéros de registres suivent la numérotation Modbus interne du MCU (base de dispatch 0x15).

Avertissement : analyse menée à des fins d'interopérabilité et de recherche sur du matériel possédé par l'utilisateur. Les adresses et comportements décrits sont spécifiques à la version de firmware analysée (V0319) et doivent être validés expérimentalement avant toute exploitation.

ADDENDUM · MISE À JOUR

Addendum — Registre 0x37 et chaîne de mesure PV

Rapport « Analyse de firmware — MCU de contrôle d'un micro-onduleur Deye » · MCU1-XXXXWG4-800W-V0319.bin (dsPIC33F)

Complément du 07/11/2026 — précise le §1 (registre 0x37) et documente la chaîne de calcul de la puissance PV (lié aux §11–12 sur le rafraîchissement).

A · Registre 0x37 (0x09F4) — caractérisation complète

Le §1 décrivait 0x37 comme « facteur d'échelle régulé par le firmware, peu fiable comme consigne ». L'analyse détaillée (13 références du binaire tracées) le précise : c'est un **coefficient de calibration pleine échelle**, et **jamais** une borne de bridage.

Caractéristique	Valeur / preuve
Variable RAM	`0x09F4 (lecture FC03 → 0x0B7A = image base 0x0B0C + 2×0x37)`
Valeur par défaut	**10000** (= gain ×1,000)
Plage acceptée	**8000 à 12000** (±20 %) ; hors plage → forcé à 10000 au boot
Stockage	EEPROM I²C 0x51, page 0x1A, octets 4–5 (big-endian) — `all.c:7118`
Bornage au boot	`if(3999 < val+0xE0C0 && val!=12000) val=10000 — all.c:7134`
Nature	gain **inverse** de pleine échelle, PAS une limite de puissance (celle-ci = reg 0x28→0A22 et reg 0x35→0A24)

Effet de l'échelle (gain inverse)

Le registre est au dénominateur : `P_scaled = P_mesurée × 10000 / reg0x37`.

reg 0x37	Facteur appliqué	Lecture courant / puissance PV
`8000`	×1,250	+25 %
`10000 (défaut)`	×1,000	référence
`12000`	×0,833	−16,7 %

Ce qu'il pilote réellement

Le même coefficient garde cohérents télémétrie et régulation ; il dérive deux consignes au moment de la (re)configuration par code-pays :

- **0x09F0 = référence de courant** : `val×211>>7 ÷ 78|75` (diviseur selon tension nominale 230/220 V) — `all.c:711–718`. ≈211 au défaut. Consommée par la boucle de courant PWM (ISR bas niveau).
- **0x0A04 = échelle de puissance** (« unités par 1 % ») : `val×572>>8 ÷ 39` — `all.c:721–723`. ≈572 au défaut. `×100` = pleine échelle 100 % + seuil de surpuissance (cible de la boucle de puissance active, `all.c:4499–4528`).
- **Courant PV reporté** : via `0x0A9E = 0x0A8E × 10000 / reg0x37` (`all.c:1189–1196`), qui sert à calculer le courant des registres Modbus 110/112/114/116.

> **Conclusion.** reg 0x37 est le coefficient d'étalonnage pleine échelle du sous-système puissance/courant PV : 16 bits, défaut 10000 (×1,00), réglable 8000–12000, en EEPROM, re-borné à chaque démarrage. Le modifier décale simultanément courant, puissance et consignes de régulation — d'où le garde-fou matériel à ±20 %. Ce n'est pas un réglage utilisateur de puissance.

B · Registres PV (tension / courant par MPPT)

Cartographie extraite du constructeur d'image FC03 `FUN_rom_0058d6` (désassemblage, rom:005c00–005c3a). Le PV est exposé uniquement en **tension + courant entrelacés pour 4 entrées MPPT** ; le 800 W ne peuple que les premières.

Reg	Hex	Grandeur	Source	Unité
109	0x6D	Tension PV1	`0966[0]`	0.1 V
110	0x6E	Courant PV1	`096e[0]`	0.1 A
111	0x6F	Tension PV2	`0966[1]`	0.1 V
112	0x70	Courant PV2	`096e[1]`	0.1 A
113–116	0x71–0x74	Tension/Courant PV3, PV4	`0966/096e[2..3]`	0.1 V / 0.1 A

> **Aucun registre de puissance PV.** Le constructeur d'image n'écrit que les registres 0–493 ; rien dans 510–683 (les « DC Power PV1..4 » 672–683 de la doc triphasée ne sont pas produits ici et seraient hors du chemin de lecture plat, borné à reg ≤ 509). La puissance PV se calcule côté client : `P = V × I` (reg 109×110, …). Le calcul interne (all.c:1437–1457) confirme : `I = P_interne × 100 / V`, donc le courant est *dérivé* de la puissance interne — jamais la puissance exposée telle quelle.

C · Chaîne de calcul de la puissance PV (complète)

Tracée de bout en bout dans le binaire. Elle explique au niveau de l'instruction le rafraîchissement lent documenté aux §11–12 : **la puissance PV est une moyenne glissante sur ~10 min**.

```
v, i (échantillons ADC + gains calibration)
   → mul.ss  v·i                    [rom:000a36]   ← le V×I (produit signé par échantillon)
   → Σ v·i  dans struct mesures 0x0EFA (+0x7a / +0x12a)
   → ÷N + ×20 + décalages        [rom:000a02/a26/a38]   (échelle point-fixe #1)
   → 0x0F7C / 0x0F80 = P moyenne/cycle par voie (struct 0x0EFA +0x82 / +0x86)
   → ×2, intègre par cycle          [rom:000972]   → accumulateur 0x08D4 / 0x08D8
   → latch ~60 s + reset             [ISR Timer4 rom:001f98] → 0x08DC / 0x08E0
   → fenêtre glissante ~10 min        [rom:0056dc, ring 0x0AAE/0x0AD6]
   → ÷2 ÷(30000|36000) ÷75         [rom:005720]   (échelle point-fixe #2 ; ÷75 = coeff final W)
   → 0x0A8E / 0x0A92 = PUISSANCE PV par voie, en W (moyenne 10 min)
   → ×10000 / reg0x37                [all.c:1189]   → 0x0A9E
   → courant PV → registres Modbus 110 / 112 / 114 / 116
```

Étage	Adresse	Rôle
Produit V×I	`rom:000a36`	MAC signé des échantillons, struct par voie 0x0EFA
P moyenne/cycle	`0x0F7C / 0x0F80`	résultat MAC normalisé (÷N), offset +0x82 / +0x86
Intégration	`rom:000972`	accum += 2×P, par cycle réseau, si string activé (0xEE8/0xEE9)
Latch slot	`rom:001f98`	ISR Timer4 ~60 s : fige + remet à zéro
Moyenne 10 min	`rom:005720`	÷2 ÷(30000 50 Hz|36000 60 Hz) ÷75 → watts
Calibration	`all.c:1189`	×10000/reg0x37 (voir §A)

> **Note d'échelle.** Deux facteurs 2 (×2 à l'intégration, ÷2 à la moyenne) s'annulent — mise à l'échelle Q interne, pas un gain physique. Le coefficient absolu ADC→watts est réparti sur deux étages : le `×20`+décalages (rom:000a38) et le `÷75` (rom:005720) ; `÷30000|36000` n'est que la longueur de la fenêtre temporelle.

Limite de l'analyse statique

Le **format Q exact** du MAC (quel offset de la struct 0x0EFA est *v* brut, *i*, ou Σv·i, et l'échelle de chaque division) n'est pas épinglé : le décompilateur n'est pas disponible pour ce cœur dsPIC dans ce module Ghidra, et le traçage manuel de l'ISR serait long et fragile. La valeur absolue en watts se confirmerait **en live** (comparer `0x0A8E × reg0x37` à une puissance PV connue). Le reste de la chaîne (structure, adresses, étages d'échelle) est prouvé.

Addendum technique — rétro-ingénierie à des fins d'interopérabilité sur matériel possédé par l'utilisateur. Scripts : FindRefs.java, RefsTo.java, AsmRange.java (C:\Tools\dspic-mw4c\scripts).

ADDENDUM 2 · MISE À JOUR DU FIRMWARE

Addendum 2 — Reflash du MCU, protocole d'update et sécurité

Rapport « Analyse de firmware — MCU de contrôle d'un micro-onduleur Deye » · MCU1-XXXXWG4-800W-V0319.bin (dsPIC33F) + clé WiFi MW4C_14_CSIP_1.14.bin (Andes NDS32)

Complément du 07/11/2026 — répond à « peut-on reflasher le MCU principal, et comment via Solarman ? » : mécanisme, protocole AT/V5, format d'image + CRC, analyse de sécurité, outillage de reproduction.

E · Le MCU principal est-il reflashable ? OUI, en service

Le firmware du dsPIC se met à jour sur le terrain (option proposée par le serveur web de la clé WiFi). Point clé : **ce n'est pas l'application dsPIC qui se grave elle-même**. L'image analysée ne contient aucun code d'écriture flash ; la gravure est faite par un **bootloader résident** absent du fichier distribué.

Observation (toute l'image rom:0–0x8bff)	Preuve	Conséquence
Aucune écriture flash	`tblwt = 0`	l'appli ne programme pas la flash
Contrôleur flash jamais touché	`NVMCON 0x760 = 0, NVMKEY 0x766 = 0`	pas de séquence RTSP
Pas de bootloader embarqué	`reset (rom:0) → goto 0x44d2 = init appli`	le bootloader est ailleurs (haut de flash, hors image OTA)
Aucun saut hors image	les 2 « OOB » (rcall 0xd0c6, goto 0x3c0000) sont de la **donnée mal désassemblée** (PA 0x7000–0x7600)	l'appli ne saute pas vers un bootloader
Lecture flash seule	`tblrd = 7, uniquement au boot (0x25e–0x2ba)`	copie des données init flash→RAM (runtime C)

> **Entrée en bootloader.** L'appli n'a pas de « commande magique » d'update (pas de comparaison au start-byte 0xA5, les 3 `reset` sont des trap/fault). Elle a en revanche un **watchdog de communication** : compteur `0x09A8` (incrémenté en `rom:00432e` quand le dialogue se dégrade, plafonné à 50) → si > 10, `reset` (`rom:00468e`). Modèle retenu : le bootloader résident tourne au reset et écoute l'UART ; le module déclenche la MAJ en provoquant un **reboot** du MCU.

F · Qui orchestre la mise à jour (via Solarman)

Le dsPIC est **100 % passif** (aucune pile réseau). Toute la logique est dans la clé WiFi (module NDS32), pilotée par le cloud. Chaîne complète :

```
[CLOUD Solarman] --TCP V2 clair--> [MODULE WiFi]
   1. pousse une commande AT (handler DTU_Wifi_Event, type ctrl 0x01)
   2. AT+INVUPURL / AT+SUPDATE  =  URL HTTP + CRC32 attendu
   3. GET http://.../MCU1-...bin           (FUN_ram_30051ae4)
   4. staging en SPI flash du module 0x300f1000  (+ descripteur YZ_FILE_UPLOAD_OK + taille + CRC32)
   5. REBOOT du dsPIC
[MODULE] --UART, trames V5--> [dsPIC bootloader résident]
   6. KIT_INFO (taille, nb blocs) -> blocs de 512 o -> finalize
   7. le bootloader grave la flash, calcule son CRC32, le renvoie (Device_CRC32_Val)
   8. le module compare -> "upload file yz_inverter_updata ok"
```

Deux firmwares distincts sont gérés par la même UI web : **firmware module** (OTA interne, chemin `update_software`+MD5) et **firmware onduleur/MCU** (chemin ci-dessus). Déclenchement cloud ou local (voir §G) ; sur le serveur Deye `http://47.254.36.66` (HTTP clair), auto-update ~30 min après mise en ligne.

G · Interface AT et trame de transport V5

Accès AT local — UDP port 48899

Poignée de main (confirmée par l'outil `s10l/deye-logger-at-cmd`) : envoyer `WIFIKIT-214028-READ` → réponse `<ip>,<mac>,<mid>` → envoyer `+ok` → puis `AT+<cmd>\n`, réponse `+ok=<val>`. Bonus : `AT+INVDATA=<len>,<trame_hex>` = tunnel Modbus RTU vers l'onduleur (slave 01, FC03/FC10, CRC16 0xA001).

Commandes d'update (parseur `FUN_ram_300524e8`)

Commande AT	Flag 0x5420	Cible	URL dans…
`AT+UPURL=<url>`	2	firmware du module WiFi (update_software+MD5)	`word[1]`
`AT+INVUPURL=<url>`	1	onduleur/MCU, sans CRC32	`word[1]`
`AT+WEBUPURL=<url>`	3	fichiers web du module	`word[1]`
`AT+SUPDATE=…`	4	**onduleur/MCU avec CRC32** (flash vérifié)	`word[2]+word[3]`

Tokenizer `FUN_ram_30030764` → délimiteurs `= , espace CR LF`, ≤10 champs. Pour le mode 4 :

```
AT+SUPDATE=<w1>,http://<host>,/<chemin/fichier.bin>,<w4>,<CRC32 décimal>
            word2 = scheme/host (test "http:")   word5 = CRC32 (strtol base 10 -> gp+0x54a0)
            word3 = chemin (URL = http://%s%s)    w1/w4 = rôle exact non confirmé
```

Trame V5 (Solarman) — triple-confirmée (RE module + dummycloud Hypfer)

```
[0]   0xA5                magic
[1-2] longueur payload     uint16 LE
[3]   unknown1 (0x10)
[4]   type                 0x41 HANDSHAKE/KIT_INFO | 0x42 DATA | 0x43 WIFI | 0x47 HEARTBEAT
[5]   msgID réponse       [6] msgID requête (compteur)
[7-10] n° de série logger  uint32 LE
[11…] payload             (firmware : blocs de 512 o)
[n-2] checksum             somme 8 bits des octets [1 .. n-3]
[n-1] 0x15                 fin
```

Séquence firmware : **cmd 2 (KIT_INFO**, annonce taille + nb de blocs gp+0xb299) → **cmd 1 (bloc 512 o**, indexé gp+0x4f73) → **cmd 0x81 (finalize+CRC32)**. Cadence ~1 s/trame.

H · Format d'image et checksum RÉSOLU

Le checksum n'est **pas** celui de `FUN_ram_3004c858` (qui s'avère être une simple **somme 32 bits** de mots LE — utilisée seulement pour le firmware *du module*). Pour l'image onduleur, le désassemblage + rejeu confirment :

Élément	Valeur
Trailer (8 derniers octets, **big-endian**)	`[longueur uint32][CRC32 uint32]`
Algorithme CRC	**CRC32 zlib/PKZIP standard** (poly 0xEDB88320, init/xorout 0xFFFFFFFF, réfléchi)
Périmètre	les `longueur` premiers octets, `[0 : longueur]`
MCU1-XXXXWG4-800W-V0319.bin	longueur = 71669 (0x000117F5), CRC32 = **0xA0AFBEE7**
Valeur pour AT+SUPDATE (décimal)	**2695872231**

> **Conséquence.** Le CRC étant un zlib standard **recalculable**, une image modifiée peut être rendue valide : modifier le payload, réécrire le trailer `[longueur BE][CRC32 BE]` avec `zlib.crc32(payload)`, et passer ce même CRC (décimal) dans `AT+SUPDATE`. Vérifié sur l'image d'origine (calcul == valeur stockée).

I · Analyse de sécurité de la chaîne d'update

La chaîne complète est **sans authentification cryptographique**. La seule crypto du module est celle du Wi-Fi (WPA2 : eapol/TKIPAES/sha1ssl/hmac) — rien pour authentifier le serveur ni signer le code.

Maillon	Protection attendue	Réalité
Serveur → module (Solarman V2)	TLS + auth	**TCP en clair** (TLS désactivé)
Téléchargement du .bin	HTTPS + cert	**HTTP clair** (http:// ×8, https:// ×0)
Intégrité de l'image	signature (RSA/ECDSA)	**CRC32 seul**, fourni par la commande
Authenticité du code	clé publique embarquée	**aucune** (pas de RSA/ECDSA/X509)

> **Vecteurs.** (1) **Usurpation du serveur** Solarman (DNS/ARP spoofing, rogue AP) → pousser un `AT+INVUPURL`/`AT+SUPDATE` vers une image arbitraire. (2) **Injection AT locale** (UDP 48899, UI web, série) — plus simple, et voie légitime pour l'interop sur matériel possédé. Barrières

réelles : se placer sur le réseau, et produire une image bien formée (format §H, connu).

Nuances. Depuis ~août 2024, un firmware récent de la clé introduit **TLS** (casse le mock cloud et ModbusTCP) — vérifier via `AT+VER`/`AT+BVER` ; le 48899 local reste a priori utilisable. La présence d'une signature dans le **bootloader dsPIC** n'est pas vérifiée (bootloader absent du dump) ; toutes les couches visibles étant CRC32-only, elle est *probablement* absente.

J · Reproduction / outillage

- **Serveur HTTP de firmware sur ESP32 (ESPHome)** : external component utilisant les classes natives d'ESPHome (`web_server_base` + `web_server_idf::AsyncWebServer`, via `AsyncWebHandler`), **sans bibliothèque externe** ; image embarquée en flash, servie à l'URL du logger. (`deye_fw_server/`)
- **gen_fw_header.py** : convertit le .bin en header C et calcule/vérifie le CRC32 Deye (zlib sur `[0:longueur]`).
- **Ressources externes** : `Hypfer/deye-microinverter-cloud-free` (mock cloud `dummycloud`, TCP 10000 ; redirection via `/config_hide.html` « Cloud Server A/B »), `s10l/deye-logger-at-cmd` (AT sur 48899), `dasrecht/deye-firmware` (images : `0_5408_15/MCU1-XXXXWG4-800W-V0319.bin`, digests SHA-256).

Addendum 2 — rétro-ingénierie à des fins d'interopérabilité et de recherche de sécurité sur matériel possédé par l'utilisateur. Projets Ghidra : C:\Tools\dspic-mw4c (dsPIC), C:\Tools\nds32-mw4c (module NDS32). Scripts : ScanNVM, OobScan, CallerTrace, RefsTo, AsmRange, ListAsm.

ADDENDUM 3 · FIRMWARE LOGGER & OUTILLAGE

Addendum 3 — Flash du logger WiFi, outillage ESP32 et suivi

Rapport « Analyse de firmware — MCU de contrôle d'un micro-onduleur Deye » · clé WiFi MW4C (Andes NDS32) + MCU dsPIC33F

Complément du 08/11/2026 — prolonge l'Addendum 2 : format et checksum de l'image du **logger**, outillage ESP32/ESPHome consolidé, et ce qu'on peut observer de la **progression** d'un flash.

K · Flash du firmware du logger (clé WiFi)

Contrairement au MCU, le logger met à jour **sa propre** flash (il a un bootloader, cf. `AT+BVER`) — voie prévue par le constructeur, plus sûre. Déclenchement : page **« Upgrade Firmware »** de l'UI web, ou `AT+UPURL=<url>` (mode 2) puis `AT+OTA`.

Format de l'image logger (confirmé)

En-tête de 256 octets + payload brut (pas de LZMA pour le MW4C). Ex. : `MW4C_14_CSIP_1.14.bin`.

```
+0x00  "HF-LPD100X Image"      magie ASCII (16 o)
+0x10  longueur payload        u32 LE  (0x00077510 = 488720)
+0x14  CRC                     u32 LE  -- SEUL champ verifie a l'OTA
+0x18  0x00000000
+0x1C  longueur (repetee)      u32 LE
+0x20  2e checksum             u32 LE  -- non verifie a l'OTA
+0x24..0xFF  zeros             (puis payload a partir de 0x100)
```

Validation OTA et algorithme de CRC

Routine `FUN_ram_300397bc` (« check_upgrade_image_ok ») :

```
si  memcmp("HF-LPD100X Image", img, 16) == 0
et  FUN_ram_3002d810(img+0x18, longueur+0xE8, 0) == *(u32*)(img+0x14)   -> image OK
```

`FUN_ram_3002d810` est un **CRC32 calculé par demi-octets (nibbles)**, table de 16 entrées `@0x3006d440` (= polynôme réfléchi standard `0xEDB88320`), **init = 0, SANS xor final**. Plage : `[0x18 : fin de fichier]` (soit 0xE8=232 octets d'en-tête à partir de 0x18, puis tout le payload).

> **Vérifié.** Ce calcul reproduit exactement le champ `+0x14 = 0x28E011B6` de `MW4C_14_CSIP_1.14.bin`. Le checksum étant reconstructible, des **images logger modifiées** sont possibles (outil `mw4c_header.py --rebuild` : fige l'en-tête puis recalcule `+0x14`).

MCU vs logger — deux checksums différents

	MCU (dsPIC)	Logger (clé WiFi)
Commande AT	`AT+SUPDATE (mode 4)`	`AT+UPURL + AT+OTA (mode 2)`
Intégrité	trailer [len][CRC32], **CRC32 zlib standard** (init/xor 0xFFFFFFFF) sur `[0:len]`	en-tête HF-LPD100X, **CRC32 nibble init=0 sans xor** sur `[0x18:fin]`
Valeur (image d'origine)	`2695872231 (0xA0AFBEE7)`	`0x28E011B6`
Grave	bootloader résident du dsPIC (via UART)	le module grave sa propre flash
Risque	**élevé** (brique = onduleur muet, ICSP requis)	**plus faible** (bootloader/recovery ; clé briquée ≠ onduleur mort)

L · Outillage ESP32 / ESPHome (projet « deye_combo »)

Un seul ESP32 fait serveur de firmware **et** client AT, 100 % classes ESPHome, sans bibliothèque externe.

Brique	Rôle
composant `file` (jesserockz)	embarque les `.bin` (MCU + logger) en tableaux d'octets à la compilation
external component `deye_fw_server`	serveur HTTP (TCP 80) greffé sur `web_server_base` ; une instance par fichier (MULTI_CONF)
external component `deye_at`	client AT UDP 48899 (sockets lwIP) : handshake `WIFIKIT-214028-READ`, envoi de commandes AT
boutons / service HA	`FLASH MCU` (AT+SUPDATE), `LOGGER 1/2+2/2` (AT+UPURL/AT+OTA), `AT+VER/BVER/H`, service `send_at`

Pas de conflit : serveur = TCP 80, client AT = UDP 48899. Scripts hôte : `deye_crc.py` (CRC MCU) et `mw4c_header.py` (en-tête logger).

M · Suivi de la progression d'un flash

Question légitime : peut-on voir l'avancement ? Réponse nuancée selon la phase.

Phase	Observable ?	Comment
Téléchargement HTTP (serveur→logger)	**OUI**	le serveur ESP32 voit le `GET` et les octets servis ; on peut journaliser début / fin / %
Flux module→dsPIC (UART privé)	**NON (externe)**	lien interne ; le module journalise en interne (index de bloc `0x4f73`, `Send_Data_Num`, `upload file` `yz_inverter_updata ok/err`) — visible seulement sur son port de debug
Auto-flash du logger (AT+OTA)	**INDIRECT**	pas de % exposé ; fin = reboot + changement de version

> **Ce qu'on peut faire côté outil.** (1) Instrumenter le handler HTTP pour reporter un **% de téléchargement** (réponse par morceaux avec rappel d'offset). (2) **Scruter l'état** après déclenchement : interroger périodiquement `AT+VER` (logger) ou le registre de version de l'onduleur via `AT+INVDATA` (Modbus FC03) pour détecter « terminé » (nouvelle version active). (3) Lire les **logs** (`esphome logs`) : GET servi + réponses AT.

> **Limite de fond.** Le pourcentage « réel » de gravure du dsPIC n'est pas accessible depuis l'ESP32 : il faudrait soit se brancher sur le lien de debug du module, soit que le MCU/module expose un registre de progression (non identifié à ce stade). Le signal fiable de bout en bout reste : « téléchargement vu » puis « nouvelle version lue ».

Addendum 3 — rétro-ingénierie à des fins d'interopérabilité et de recherche de sécurité sur matériel possédé. Routines clés : FUN_ram_300397bc (check image), FUN_ram_3002d810 (CRC32 nibble), table 0x3006d440. Projet ESP32 : deye_combo (composants deye_fw_server + deye_at).

Addendum v6 — 8 novembre 2026

Rapport d'analyse firmware MCU micro-onduleur Deye MW4C (G4 800 W)

Ajouts : (15) risque de brick au flash du dsPIC — analyse du bootloader ; (16) outils ESPHome développés (flash + composant *deyemi_fast* + intégration).

---

15. Risque de brick au flash du MCU dsPIC — analyse du bootloader

Question traitée : en flashant le firmware applicatif patché (*_FULL.bin*), quelle est la probabilité de rendre le MCU principal (dsPIC33F) inutilisable ? La réponse repose sur une question structurelle : **peut-on toujours re-flasher l'original ?** L'analyse statique ci-dessous (Ghidra 12.1, projet *proj_dspic*) la tranche en grande partie.

15.1 — Chaîne de flash (bootloader résident)

Le MCU n'a **pas d'ICSP confirmé** accessible. La mise à jour passe par le module WiFi MW4C : le module télécharge l'image (URL Solarman + CRC32), la stocke en flash SPI, puis la pousse au dsPIC sur l'UART interne où un **bootloader résident** grave l'application et renvoie son CRC32 (Device_CRC32_Val). Côté ESP, la commande est `AT+SUPDATE=1,<url>,0,<crc>`.

15.2 — Preuve statique : l'application ne peut pas écrire la flash

Sur dsPIC, programmer la flash programme exige impérativement des instructions `TBLWTL/TBLWTH` plus la séquence de déverrouillage `NVMKEY` (0x55 / 0xAA) et le bit WR de `NVMCON`. Le scan exhaustif de l'image applicative donne :

```
ScanNVM (Ghidra, image entière rom:000000-008bff)
  tblwt  = 0      <- AUCUNE instruction d'ecriture flash
  nvmcon = 0
  nvmkey = 0
  tblrd  = 7      (lectures de constantes, benignes)
```

Conclusion ferme : l'application patchée ne contient **aucun** code capable d'écrire la flash programme. La gravure est donc effectuée par un bootloader situé dans une **région séparée, absente de l'image OTA**. Le bloc mémoire chargé (rom:000000–008bff, soit 0x11800 octets fichier) ne couvre que l'application ; la flash physique du composant est plus grande.

15.3 — Vecteur de reset et propriété du bootloader

Désassemblage du début d'image :

```
rom:000000 : lnk  #0x0      <- PAS un 'goto' vers l'application
rom:000002 : nop
rom:000004 : goto 0x0044d2   (table d'interruptions)
```

Le slot de reset (adresse 0x0) **ne contient pas de saut vers l'application**. C'est cohérent avec un bootloader qui **possède le chemin de reset** : il s'exécute en premier à la mise sous tension puis transfère le contrôle à l'application. Indice fort (pas preuve absolue, le binaire du bootloader n'étant pas dans l'image) que le bootloader est **ré-entrable au power-up**, donc atteignable même application morte.

15.4 — Scénarios de risque et récupérabilité

Deux garde-fous CRC protègent le transfert : le module vérifie le CRC32 au téléchargement, et le bootloader le revérifie avant de valider. Un transfert corrompu est **rejeté, pas commité**.

Scénario	Issue
App patchée boote mais bug (ex. mauvais registre)	Recuperable : reflash. Patches chirurgicaux, ne touchent ni boot ni UART ni update.
Transfert corrompu / mauvais CRC	Rejete par double garde CRC ; jamais commite.
Ecrasement du bootloader	IMPOSSIBLE : hors image OTA, region separee.

Coupure secteur en plein write ET bootloader non re-entrable au power-up	Seul cas potentiellement fatal (hypothese defavorable, non averee).

Bilan : probabilité de brick **faible**, et irréversibilité confinée à un coin étroit (coupure secteur pendant la gravure, sur un chip dont le bootloader ne s'auto-entrerait pas). Tout le reste est récupérable car le bootloader survit à chaque flash et l'original reste re-poussable (CRC original 0xA0AFBEE7 / déc. 2695872231 ; image patchée _FULL.bin CRC 0x1D3120C8).

15.5 — Inconnu résiduel et protocole de dé-risquage

Seul point non tranché : le bootloader se ré-entre-t-il **automatiquement au power-up** (→ même une app morte ou une coupure en plein write est récupérable) ou **seulement sur commande à l'app qui tourne** (→ dépend d'une app saine). Le binaire du bootloader n'étant pas dans l'OTA, la levée définitive passe par :

```
1. Analyser le cote HOTE du protocole dans le firmware du logger
   (MW4C_14_CSIP_1.14.bin) : reset-t-il le dsPIC vers un
   bootloader au power-up, ou envoie-t-il une commande a l'app ?
2. Banc : premier flash = ORIGINAL -> ORIGINAL (no-op reversible),
   puis _FULL.bin, puis verifier qu'on peut reflasher l'original.
3. Alimentation stable, onduleur sans soleil (0 production) pendant
   toute la sequence ; ne rien debrancher.
```

16. Outils ESPHome développés

Trois briques logicielles ont été produites autour de cette analyse : (16.1) le flash des deux firmwares depuis un ESP32, (16.2) l'image dsPIC patchée, (16.3) un nouveau composant ESPHome pour la télémétrie rapide, (16.4) sa configuration d'intégration.

16.1 — Flash des firmwares (projet *deye_combo*)

Config ESPHome `deye-combo.yaml` + composants locaux `deye_at` (client AT sur UDP 48899, handshake WIFIKIT) et `deye_fw_server` (serveur HTTP qui sert le .bin). L'ESP héberge l'image et pilote le module via boutons :

```
# MCU dsPIC (mode SUPDATE : URL + CRC)
AT+SUPDATE=1,http://<ip_esp>,/0_5408_15/MCU1-...-V0319.bin,0,<crc>

# Logger MW4C (mode OTA module)
AT+UPURL=http://<ip_esp>/0_D0002_18/MW4C_14_CSIP_1.14.bin
AT+OTA
```

Note : le `MW4C_...bin` embarqué est le firmware **d'origine** du logger (pour reflash/récupération), pas une version modifiée. **Seul le dsPIC est modifié.**

16.2 — Image dsPIC patchée *_FULL.bin*

11 patchs, 29 octets modifiés, CRC recalculé **0x1D3120C8** (zlib/BE sur 71669 octets). Rollback = image d'origine, CRC déc. 2695872231.

```
P1  0 ecriture EEPROM energie  : NOP @0xAA24 / @0xAC6C (pages 0x1B/0x1C)
P2  reg 0x35 libre, borne 100% : @0x7A84 / @0x7A8C / @0x7AC8 (decouple 0x28)
P3  P_AC instantanee 32 bits   : reg93(bas)@0xB770 + reg100(haut)@0xB78C  <- 0x08a8
P4  V/P PV par string rapides  : reg25/39 (V<-0x0f1e/0x0f20) @0xB37C/0xB450
                                 reg47/103 (P<-0x0f7c/0x0f80) @0xB4B8/0xB7D8
Conversion : V_PV = reg/16 ; P = reg x k ; I = P / V (cote ESP32)
```

16.3 — Nouveau composant *deyemi_fast*

Fork du composant `deyemi` (même transport Solarman V5, même carte native et chemins de commande ; namespace/classe/platform renommés pour coexister). Il ajoute un bloc `fast:` qui décode les 6 registres repointés par le firmware patché — tous dans le poll 0x0001–0x007D existant, donc **zéro trafic réseau supplémentaire** — et les publie à la cadence `poll_interval` (2–3 s). **Compile OK** (esphome compile, ESP-IDF). Emplacement : `components/deyemi_fast/`.

Auto-calibration de k (aucune valeur magique)

Les valeurs brutes sont en unités internes ; watts = k × brut. k est une **constante firmware** (le trim ADC par appareil s'annule car le registre watts moyenné natif et la valeur instantanée partagent la même chaîne ADC). Le composant la **mesure lui-même** : en régime stable, k = watts_natifs / brut. k est **persisté en flash** (par onduleur) et publié via `fast.k`.

```
Gardes derivees du firmware (automatiques) :
  x2 mono/bi-string (all.c:4260-4271) : annule + calib seulement si 2 strings
  lag moyenne 10 min : calib exige raw stable pendant calibration_settle_time (900s)
  overflow : total sur 0x08a8 32 bits ; warn si troncature mot-bas per-string
Fix herite : format_hex_pretty(vector) supprime dans ESPHome 2026.11-dev
  -> remplace par log de taille (meme bug dans le deyemi d'origine)
```

16.4 — Intégration *deye_telemetry.yaml*

Config prête (validée `esphome config`). Capteurs natifs lents conservés (fréquence, température, énergies — seules sources) + capteurs rapides, + pilotage 0x35 (RAM, 0 usure).

```
deyemi_fast:
  - id: deye1
    ip_address: <ip_logger>   # IP locale du MW4C
    ip_port: 8899             # Modbus-TCP local (pass-through V5)
    sn: <sn_LOGGER>           # serie du LOGGER, pas de l'onduleur !
    model: deye_2mppt
    poll_interval: 3s
sensor:
  - platform: deyemi_fast
    deyemi_fast_id: deye1
    ac: { voltage: {...}, frequency: {...} }     # lents, natifs
    fast:
      ac: { power: {...}, current: {...} }       # instantanes
      k:  {...}                                  # diagnostic
      dc_channels:
        - pv0: { voltage: {...}, power: {...}, current: {...} }
        - pv1: { voltage: {...}, power: {...}, current: {...} }
```

Prérequis : firmware patché _FULL.bin flashé (sinon les `fast.*` restent indisponibles et k ne converge jamais), port 8899 du logger ouvert, un seul client Modbus-TCP à la fois. Le logger reste en firmware d'origine : **aucun firmware logger modifié n'est requis** pour la télémétrie (le cloud est un sujet découplé).

Addendum v7 — 8 novembre 2026

Rapport d'analyse firmware MCU micro-onduleur Deye MW4C (G4 800 W)

Ajouts : (17) *deyemi_fast* rendu compatible 4 MPPT (registres configurables) ; (18) acquisition de firmware & schéma OTA (pas de dump AT, URL cloud décodée, sonde AT+MTYPE).

---

17. *deyemi_fast* rendu compatible 4 MPPT

Ce firmware/patch couvre un **2 MPPT** (800 W, 2 strings). Les versions **4 MPPT** (M130–M220) ont 4 strings. Le composant *deyemi_fast* a été rendu **agnostique du firmware** pour les couvrir sans recompilation :

• jusqu'à **4 canaux rapides** (tableaux de capteurs en [4], `fast.dc_channels` max 2→4) ;
• **adresses de registres configurables en YAML** (défauts = le 2 MPPT validé) ;
• total AC : agrégat `0x08a8` (×2 mono-string, 2 MPPT) **ou** somme des puissances par string (modèle-agnostique) quand l'agrégat n'est pas fourni.

```
deyemi_fast:
  - id: deye1
    # 4-MPPT : un registre par string (A..D), du firmware patche de CE modele
    fast_pv_voltage_registers: [25, 39, 71, 73]
    fast_pv_power_registers:   [47, 103, 110, 111]
    # omis -> total AC = somme des puissances par string
```

Statut : compile OK (2 MPPT) + `esphome config` OK (4 MPPT). **Bloqué** : le firmware 4 MPPT (binaire non possédé) — les adresses des strings C/D sont inconnues tant qu'un firmware 4 MPPT n'a pas été rétro-analysé. Nativement, la carte expose déjà PV1–PV4 en lent (0x6D–0x74).

18. Acquisition de firmware & schéma OTA

18.1 — Pas de « dump » par commande AT (vérifié)

Revue des ~130 commandes AT du binaire module (`MW4C_14_CSIP_1.14.bin`) : celles qui touchent au firmware ne font qu'**écrire** (OTA) ou **lire la config**, jamais lire la flash programme. `AT+SUPDATE`/`AT+OTA`/`AT+UPURL` = téléversement ; `AT+CFGRD` = paramètres ; « DUMP » = `AT+RF_DUMP` (registres radio). **Aucun read-back de flash**, ni MCU ni module. Le dsPIC est gravé par un bootloader à sens unique (renvoie un CRC32, pas le contenu) et sa code-protection bloque la lecture ICSP.

18.2 — Mécanisme OTA (Hi-Flying 2 étages, customisé Solarman)

Le module (1) rapporte au serveur son MAC + **deviceType** + versions ; (2) télécharge un manifeste `http://<hôte>/config.txt` ; (3) ce manifeste liste, par composant, les URLs des fichiers (software / config / web) avec nom, taille, type et CRC ; (4) il télécharge chaque fichier, vérifie le CRC et grave la cible via le bootloader. Hôtes trouvés : `data1/2.solarmanpv.com`. Composants ciblables : MCU1 (dsPIC), MCU2 (C28x), DSPM/DSPS, FUSE, WEB.

```
http://<hote>/0_<codeModele>_<type>/<fichier>.bin
  MCU (dsPIC) : 0_5408_15/MCU1-XXXXWG4-800W-V0319.bin
  Module WiFi : 0_D0002_18/MW4C_14_CSIP_1.14.bin
  0 = prefixe fixe ; codeModele (5408) = modele Solarman ;
  type (15=MCU1, 18=module) ; le prefixe du nom indique la cible.
```

Point clé : ni `5408` ni le nom de fichier ne sont dans le firmware du module — ils viennent du `config.txt` cloud, choisi par **deviceType**. On ne peut donc pas construire l'URL d'un 4 MPPT à l'aveugle : il manque son code modèle + nom de fichier, côté Solarman.

18.3 — Obtenir un firmware 4 MPPT

Approche	Coût	Ce qu'on obtient
AT identite (AT+MTYPE...) via deye_at	Pur YAML, nul	le deviceType -> requete cloud ensuite
Intercepter config.txt (DNS detourne)	Composant C++ (DNS+HTTP)	l'URL reelle + le .bin en transit

AP+STA+NAPT sur l'ESP	C++ IDF, reconfig WiFi	idem, mais plus fragile (deconseille)

L'OTA étant en **HTTP clair**, l'interception est faisable. L'intercepteur complet (serveur DNS + serveur HTTP catch-all/proxy) **ne peut pas** se faire en pur YAML (le `web_server` d'ESPHome ne gère ni routes arbitraires ni proxy, et il n'y a pas de serveur DNS/TCP générique en YAML) — c'est un composant C++. En revanche, récupérer le **deviceType** se fait en pur YAML via les commandes AT d'identité.

18.4 — Sonde AT ajoutée à *deye_combo*

Boutons ajoutés (groupe « Identité modèle »), réponses dans le text_sensor « dernière réponse AT » : `AT+MTYPE` (device type), `AT+UID`, `AT+MID`, `AT+WRMID`. Sur une unité 4 MPPT, ils doivent révéler le code modèle à recouper avec `AT+VER`/`AT+BVER` et le préfixe du nom de firmware. C'est la première étape, gratuite, avant tout intercepteur.